



西北农林科技大学
NORTHWEST A&F UNIVERSITY

西北农林科技大学信息工程学院

《面向对象程序设计实践》实习报告

3D 点云目标分割方法设计与实现

学 号: 2023012760

学 生 姓 名: 马建宇

专 业 名 称: 软件工程

班 级: 软工 2004

指 导 教 师: 耿楠 教授

实 践 日 期: 2026 年 5 月 26 日—6 月 9 日

2026 年 6 月

目录

一、综合训练目的与要求	1
二、综合训练任务	1
1. 题目与原始要求	1
2. 需求分析	1
三、开发环境与工具	2
四、总体设计	2
1. 处理流程	2
2. 模块划分	2
五、详细设计说明	3
1. 类设计	3
2. KD 树与 k 近邻	3
3. 法向量与曲率估计 (PCA)	3
4. 区域生长分割	4
5. 由粗到细的迭代分割	4
6. 三维可视化与交互	4
六、系统实现	5
1. 对称矩阵特征分解 (Jacobi 法)	5
2. 区域生长核心	7
七、调试与测试	8
1. 单元测试	8
2. 分割结果与由粗到细可视化	9
3. 性能	9
4. 已知局限	9
八、AI 使用情况	10
九、实习日志	11
十、实习总结	11
A. 附录: 程序设计及报告要求	13
1. 代码量	13
2. 功能方向	13
3. 类与文件要求	13
4. 报告与设计要求	13

B. 附录：开发工具与资源要求	15
1. 编程语言	15
2. 代码管理	15
3. 编程工具推荐	15
4. 图形界面（界面非必需，可无界面）	15
5. 其他相关开源库（选用）	15
6. 论文撰写	15
C. 附录：核心代码清单	16
1. KD 树（k 近邻查询）	16
2. PCA 法向量与曲率估计	18
3. 区域生长与由粗到细分割	20

一、综合训练目的与要求

本次综合训练以“3D 点云目标分割方法设计与实现”为题，目的在于：

(1)运用面向对象程序设计思想（类的抽象与封装、模块解耦、接口设计）完成一个具有一定难度的图形学课题，并解决其中的实际问题；

(2)掌握点云处理的基本流程：近邻搜索、法向量估计、表面分割，以及由粗到细的迭代细分；

(3)训练用 OpenGL 进行三维数据可视化与人机交互的能力；

(4)训练规范的程序设计文档撰写、单元测试设计与结果分析能力。

具体的程序设计与报告要求，详见附录第 A 节。

二、综合训练任务

1. 题目与原始要求

选题 56 属于“科研类”，原始要求为：(1) 需要一定的图形学和 OpenGL 代码基础；(2) 读取 3D 点云文件并进行绘制；(3) 对原始点云模型实现**由粗到细**的迭代分割（需考虑点云表面朝向、点的位置分布、局部形状的连通性、表面片段的一致性与差异性度量）；(4) 用不同颜色显示不同分割片段，实现对迭代分割过程各阶段的展示。

2. 需求分析

将上述要求拆解为如下可实现的功能点，如表 表 1所示。

表 1 功能需求分解

编号	功能	说明
F1	点云读写	读取 .ply (ascii / 二进制小端) 与 .xyz 文本；导出按片段着色的 .ply
F2	近邻搜索	用 KD 树支持 k 近邻查询，为法向量估计与区域生长提供邻域
F3	法向量与曲率	对每个点用 PCA 估计法向量（表面朝向）和曲率（局部平坦度）
F4	区域生长分割	按表面朝向一致性与连通性把点云分成若干光滑片段
F5	由粗到细	逐层收紧法向夹角阈值，对每个粗片段继续细分，记录各层结果
F6	三维可视化	OpenGL 绘制点云，支持旋转/缩放/平移、正侧俯三视图、点大小调节
F7	分色与回放	不同片段不同颜色；用滑块在各层间切换，回放由粗到细过程
F8	测试与数据	合成点云生成器 + 单元测试，验证各模块正确性

三、 开发环境与工具

- 语言标准: C++17; 构建系统: CMake (≥ 3.16)。
- 编译器: Apple Clang 14 (macOS 12.7); 同样适用于 GCC/MSVC。
- 图形库: GLFW 3.4 (窗口与输入) + 系统 OpenGL 3.3 核心模式 (渲染) + Dear ImGui 1.90 (控制面板)。
- 文档: $\text{N}_\text{W}^{\text{AFU}}$ Paper 模板, Xe L^{A} TeX 编译。
- 说明: 题目原文建议使用 PCL/OpenGL。考虑到课程“展示自身对算法的理解”这一目的, 以及 PCL 在本机环境下依赖沉重, 本项目**不依赖 PCL**, KD 树、PCA 特征分解、区域生长等算法均自行实现。

四、 总体设计

1. 处理流程

系统的数据处理流程如图 1所示: 从点云文件出发, 依次经过近邻搜索、法向量估计、区域生长, 得到由粗到细的多层分割结果, 最终送入 OpenGL 视口着色显示。

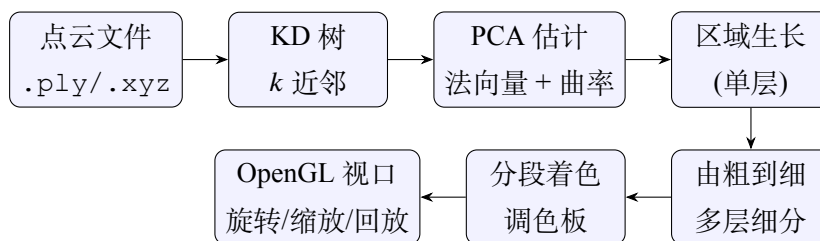


图 1 系统数据处理流程

2. 模块划分

工程划分为“核心算法库 + 应用层”两部分, 核心库不依赖任何图形界面, 可单独编译与测试:

- 核心库 pcseg_core**: Vec3、PointCloud、KdTree、法向量估计、Segmentation、PointCloudIO、合成数据生成器;
- 命令行 pcseg_cli**: 无界面跑通“读云 $\rightarrow$ 法向量 $\rightarrow$ 分割 $\rightarrow$ 导出”;
- 图形界面 pcseg_gui**: Viewer + Camera, 基于 GLFW/OpenGL/ImGui;
- 测试 pcseg_tests**: 基于 CTest 的单元测试。

五、详细设计说明

1. 类设计

按面向对象思想，把“数据、算法、显示”分别封装。主要类及其职责见表 2，类之间的关系如图 2 所示（核心库被三个应用共享，界面层依赖核心库而核心库不反向依赖界面）。

表 2 主要类与职责

类 / 模块	职责
Vec3	三维向量/点，封装加减、点积、叉积、模长、单位化
PointCloud	点云容器：坐标、法向量、曲率；包围盒、质心
KdTree	3D KD 树，建树与 k 近邻查询
SymEigen	3×3 对称矩阵 Jacobi 特征分解（供 PCA 用）
Segmentation	区域生长与由粗到细分割（SegParams/SegResult）
PointCloudIO	PLY/XYZ 读取、着色 PLY 写出
Camera	轨迹球相机：旋转/缩放/平移、三视图、视图矩阵
Viewer	OpenGL 渲染、ImGui 控制面板、串联整个流程

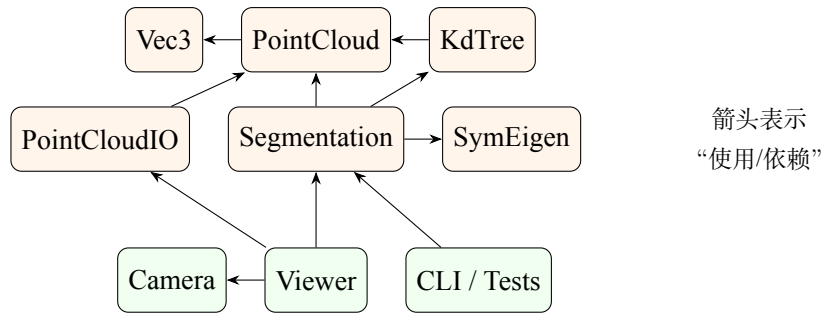


图 2 类之间的依赖关系（橙色为核心库，绿色为应用层）

2. KD 树与 k 近邻

法向量估计和区域生长都要反复查询“离某点最近的 k 个点”。若每次都遍历全部点是 $O(N^2)$ ，点数上万时很慢。KD 树按坐标轴轮流划分空间（ $x \rightarrow y \rightarrow z$ ），建树时用 `nth_element` 取中位数保证较平衡，查询时用一个大小为 k 的大顶堆维护当前最近的点，并利用“分割面距离”做剪枝，平均复杂度降到约 $O(\log N)$ 。

3. 法向量与曲率估计（PCA）

对每个点取其 k 个近邻，计算邻域质心，得到 3×3 协方差矩阵

$$C = \frac{1}{k} \sum_{i=1}^k (p_i - \bar{p})(p_i - \bar{p})^T.$$

对 C 做特征分解, 设特征值 $\lambda_0 \leq \lambda_1 \leq \lambda_2$ 。最小特征值 λ_0 对应的特征向量即为该点法向量 (邻域最“扁”的方向); 曲率定义为

$$\sigma = \frac{\lambda_0}{\lambda_0 + \lambda_1 + \lambda_2},$$

σ 越小表示邻域越平坦。 3×3 对称矩阵的特征分解用经典的 Jacobi 旋转法自行实现 (见代码 1), 不依赖外部线性代数库。

4. 区域生长分割

区域生长把“表面朝向一致、且空间连通”的点聚成同一片段, 步骤如下:

- (1) 把所有未分配点按曲率**从小到大**排序, 越平坦的越先做种子;
- (2) 取当前曲率最小的未分配点作为种子, 开启一个新片段, 做广度优先扩张;
- (3) 对当前点的每个近邻: 若其法向量与当前点法向量的夹角小于阈值 θ (一致性度量), 则并入当前片段;
- (4) 只有当该近邻自身也足够平坦 (曲率 < 曲率阈值) 时, 才把它作为新的扩张前沿——这样可以在棱边等高曲率处自然停下, 不会越过物体边界;
- (5) 重复直到队列为空, 得到一个片段; 回到 (2) 直到所有点处理完毕;
- (6) 点数小于阈值的片段视为噪声丢弃, 并把片段号压缩为连续编号。

其中, 法向夹角的判断用余弦实现: $|\hat{n}_{cur} \cdot \hat{n}_{nb}| > \cos \theta$ 即认为一致 (取绝对值是为了忽略法向量的正负朝向)。这一设计同时覆盖了题目要求的“表面朝向、位置分布、局部连通性、一致性/差异性度量”四个方面。

5. 由粗到细的迭代分割

单次区域生长只能得到一种粗细的划分。为实现“由粗到细”, 本项目采用**分层细分**:

- 第 0 层: 对整片点云用**最大的**夹角阈值 θ_{coarse} 做区域生长, 得到少数几个大块 (粗);
- 第 L 层 ($L \geq 1$): 在上一层**每一个**片段的内部, 单独用更小的夹角阈值再做一次区域生长, 把大块细分为更小的块; 阈值从 θ_{coarse} 线性过渡到 θ_{fine} 。

这样得到的各层划分是**逐层嵌套**的: 细层一定是粗层的进一步切分。每层结果都被保存下来, 界面用滑块即可回放由粗到细的全过程。

6. 三维可视化与交互

渲染基于 OpenGL 3.3 核心模式: 点坐标和颜色分别放入两个 VBO, 顶点着色器做 MVP 变换并设置点的屏幕大小, 片段着色器把方点裁剪成圆点。相机为轨迹球模型, 记录观察目标、距离、偏航角与俯仰角; 鼠标左键拖动改变偏航/俯仰, 滚轮改变距离, 右键拖动平移目标, 并提供正视、侧视、俯视三个预设视角。控制面板用 Dear

ImGui 实现, 可实时调节 k 、层数、粗/细夹角、曲率阈值、最小片段点数等参数并重新分割, 还可切换“分割结果/法向量/高度/纯色”四种着色。

六、系统实现

下面给出几处关键实现。完整代码见附录 第 C 节 与随附源码仓库。

1. 对称矩阵特征分解 (Jacobi 法)

```
1 inline SymEigen symmetricEigen(float a00, float a01, float a02,
2                               float a11, float a12, float a22) {
3     // A: 工作用矩阵, 迭代中会被逐步对角化
4     double A[3][3] = {
5         {a00, a01, a02},
6         {a01, a11, a12},
7         {a02, a12, a22}
8     };
9     // V: 累积的旋转矩阵, 最终每一列就是一个特征向量
10    double V[3][3] = {
11        {1, 0, 0},
12        {0, 1, 0},
13        {0, 0, 1}
14    };
15
16    // 最多迭代若干轮, 每轮把当前最大的非对角元素旋转消掉
17    for (int iter = 0; iter < 50; ++iter) {
18        // 找出绝对值最大的非对角元素 A[p][q]
19        int p = 0, q = 1;
20        double maxOff = std::fabs(A[0][1]);
21        if (std::fabs(A[0][2]) > maxOff) { maxOff = std::fabs(A[0][2]); p = 0; q =
22            2; }
23        if (std::fabs(A[1][2]) > maxOff) { maxOff = std::fabs(A[1][2]); p = 1; q =
24            2; }
25
26        // 非对角元素已经足够小, 认为收敛
27        if (maxOff < 1e-12) break;
28
29        // 计算旋转角, 使 A[p][q] 变 0
30        double app = A[p][p];
31        double aqq = A[q][q];
32        double apq = A[p][q];
33        double phi = 0.5 * std::atan2(2.0 * apq, aqq - app);
34        double c = std::cos(phi);
35        double s = std::sin(phi);
```



```

35 // 用旋转矩阵 J 更新  $A = J^T A J$ 
36 double Apk, Aqk;
37 for (int k = 0; k < 3; ++k) {
38     Apk = A[p][k];
39     Aqk = A[q][k];
40     A[p][k] = c * Apk - s * Aqk;
41     A[q][k] = s * Apk + c * Aqk;
42 }
43 for (int k = 0; k < 3; ++k) {
44     Apk = A[k][p];
45     Aqk = A[k][q];
46     A[k][p] = c * Apk - s * Aqk;
47     A[k][q] = s * Apk + c * Aqk;
48 }
49 // 同步更新特征向量矩阵 V
50 for (int k = 0; k < 3; ++k) {
51     double Vkp = V[k][p];
52     double Vkq = V[k][q];
53     V[k][p] = c * Vkp - s * Vkq;
54     V[k][q] = s * Vkp + c * Vkq;
55 }
56 }
57
58 // 取对角线作为特征值, V 的每一列作为特征向量
59 float eval[3] = { (float)A[0][0], (float)A[1][1], (float)A[2][2] };
60 Vec3 evec[3] = {
61     Vec3((float)V[0][0], (float)V[1][0], (float)V[2][0]),
62     Vec3((float)V[0][1], (float)V[1][1], (float)V[2][1]),
63     Vec3((float)V[0][2], (float)V[1][2], (float)V[2][2])
64 };
65
66 // 按特征值从小到大排序 (简单的冒泡, 只有 3 个元素)
67 for (int i = 0; i < 2; ++i) {
68     for (int j = 0; j < 2 - i; ++j) {
69         if (eval[j] > eval[j + 1]) {
70             std::swap(eval[j], eval[j + 1]);
71             std::swap(evec[j], evec[j + 1]);
72         }
73     }
74 }
75
76 SymEigen result;
77 for (int i = 0; i < 3; ++i) {
78     result.values[i] = eval[i];

```

```

79     result.vectors[i] = evec[i].normalized();
80 }
81 return result;
82 }

```

代码 1 3x3 对称矩阵 Jacobi 特征分解

2. 区域生长核心

```

1  std::vector<int> regionGrow(const PointCloud& cloud,
2                             const std::vector<std::vector<int>>& adjacency,
3                             const std::vector<char>& allowed,
4                             float smoothnessRad,
5                             float curvatureThreshold,
6                             int minClusterSize,
7                             int& segCountOut) {
8      const int n = (int)cloud.points.size();
9      std::vector<int> labels(n, -1);
10     const float cosThreshold = std::cos(smoothnessRad); // 用余弦比较, 省去每次
11                                                         // acos
12     // 把子集内的点按曲率从小到大排序, 越平坦的越先当种子 (区域生长的经典做法)
13     std::vector<int> order;
14     order.reserve(n);
15     for (int i = 0; i < n; ++i) {
16         if (allowed[i]) order.push_back(i);
17     }
18     std::sort(order.begin(), order.end(), [&](int a, int b) {
19         return cloud.curvature[a] < cloud.curvature[b];
20     });
21
22     int segId = 0;
23     for (int seed : order) {
24         if (labels[seed] != -1) continue; // 已经归到某个段了
25
26         // 从这个种子开始, 广度优先地向外扩张出一个段
27         std::queue<int> q;
28         q.push(seed);
29         labels[seed] = segId;
30
31         while (!q.empty()) {
32             int cur = q.front();
33             q.pop();
34             const Vec3& nCur = cloud.normals[cur];

```

```

35
36     for (int nb : adjacency[cur]) {
37         if (!allowed[nb] || labels[nb] != -1) continue;
38
39         // 当前点与邻居法向量的夹角，用 |cos| 比较（忽略法向正反方向）
40         float c = std::fabs(dot(nCur, cloud.normals[nb]));
41         if (c < cosThreshold) continue; // 夹角太大，不属于同一光滑面
42
43         labels[nb] = segId; // 并入当前段
44         // 只有足够平坦的点才继续作为新的扩张前沿，避免越过棱边
45         if (cloud.curvature[nb] < curvatureThreshold) {
46             q.push(nb);
47         }
48     }
49 }
50 ++segId;
51 }
52
53 // 统计每个段的点数，丢弃太小的段（标回 -1），再把段号压缩成连续的 0..K-1
54 std::vector<int> count(segId, 0);
55 for (int i = 0; i < n; ++i) {
56     if (labels[i] >= 0) count[labels[i]]++;
57 }
58 std::vector<int> remap(segId, -1);
59 int kept = 0;
60 for (int s = 0; s < segId; ++s) {
61     if (count[s] >= minClusterSize) remap[s] = kept++;
62 }
63 for (int i = 0; i < n; ++i) {
64     if (labels[i] >= 0) labels[i] = remap[labels[i]];
65 }
66
67 segCountOut = kept;
68 return labels;
69 }

```

代码 2 区域生长：按曲率选种子、按法向夹角扩张

七、调试与测试

1. 单元测试

项目自带轻量测试框架（用宏统计检查项与失败数），并接入 CTest。测试用例及结果见表 3，共 69 项检查全部通过。

表 3 单元测试用例

用例	验证内容	结果
Vec3	模长、点积、叉积、单位化	通过
KdTree	k 近邻结果与暴力遍历完全一致 (20 组查询)	通过
Normals	平面上的点法向量接近 (0, 0, 1)、曲率 < 0.02	通过
PLY 往返	着色 PLY 写出后读回, 坐标一致	通过
两平面	朝向不同且分离的两平面恰好分成 2 段, 纯度 > 0.95	通过
演示场景	由粗到细段数递增, 最细层 ≥ 4 段, 纯度 > 0.85	通过

2. 分割结果与由粗到细可视化

在内置演示场景 (地面 + 球 + 圆柱 + 斜面, 共 9000 点) 上运行。图 3 与图 4 对比了由粗到细的效果: 粗层 (3 段) 把斜面与地面并为一块; 细层 (5 段) 则把斜面单独分出。图 5 是法向量着色, 可见地面法向一致 (偏蓝)、球面法向连续过渡 (彩虹色), 直观验证了法向量估计的正确性。

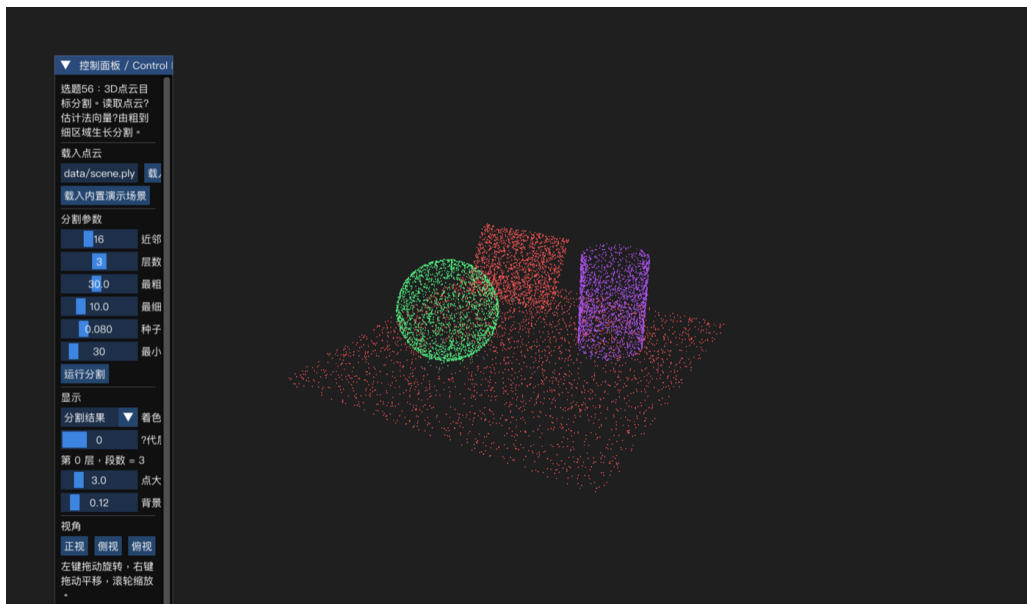


图 3 粗层分割 (第 0 层, 3 段): 斜面与地面被合并为同一片段 (红)

3. 性能

在 9000 点的演示场景上, 建 KD 树 + 法向量估计 + 三层分割合计约 80 毫秒 (Release 模式, 单线程), 交互帧率稳定。各层段数为 $3 \rightarrow 3 \rightarrow 5$, 最细层对四类目标的分割纯度约 95.5%。

4. 已知局限

纯区域生长在**相接的锐利棱边**处可能发生“泄漏”: 当两个面在棱边相连、且棱边附近法向量呈渐变时, 若中间点曲率仍低于种子阈值, 区域生长会沿渐变“绕过棱角”

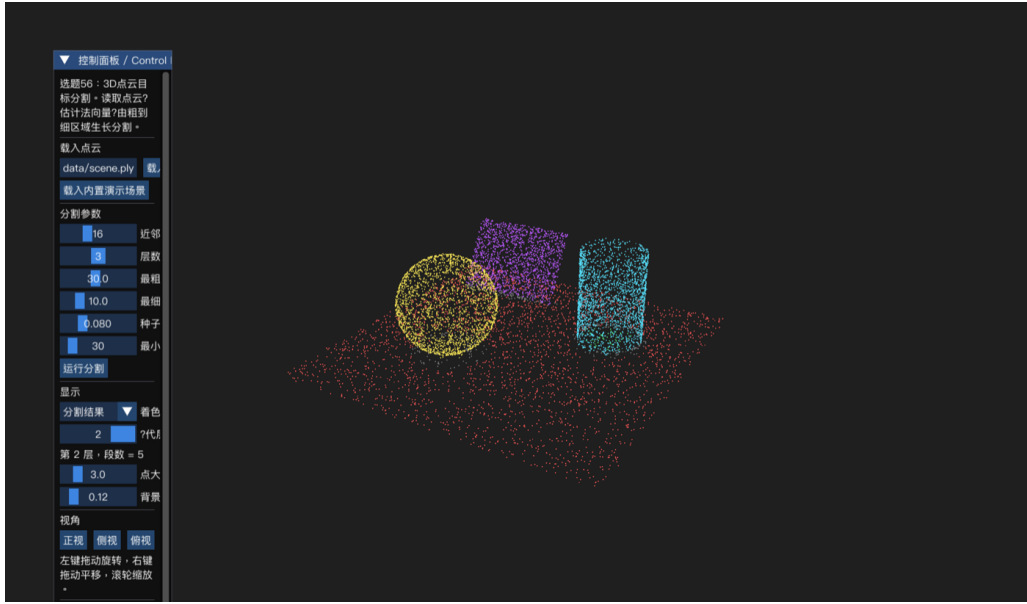


图 4 细层分割（第 2 层，5 段）：斜面（品红）已从地面中分离

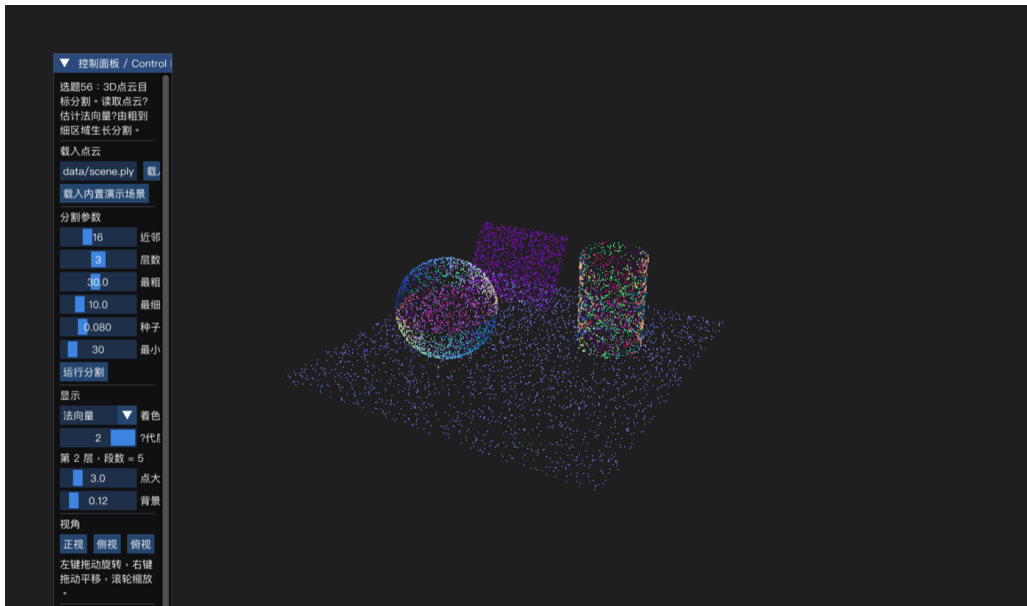


图 5 法向量着色：法向量 (n_x, n_y, n_z) 映射为 RGB

把两个面并到一起。在无噪声、两面恰好垂直相接的极端用例中即可复现。改进方向包括引入显式的边界/棱检测、或在一致性度量中加入曲率突变项。本项目通过“曲率门控种子”已显著缓解该问题，并在测试中改用**空间分离**的两平面以稳定地验证分割正确性。

八、 AI 使用情况

(1)使用环节：思路梳理（区域生长与由粗到细方案的比较）、代码编写与重构、调试（如棱边泄漏问题的定位）、单元测试设计、实习报告与 $\text{\LaTeX}$ 排版。

(2)工具：Claude Code (Anthropic 公司 Claude Opus 4.8 模型)。

(3)作用：在保证自己理解算法原理的前提下，主要用于提高编码与排版效率、快速定位编译/逻辑错误、对照检查测试用例的合理性。

(4)AI 生成代码占比：约 70%–80% 的初稿由 AI 协助生成，但所有算法选型、参数取值、测试断言和对结果的解释均经本人理解、核对与修改后确定。

(5)遇到的问题与思考：例如最初把“两垂直相接平面应分成 2 段”写成硬性断言，实际却因棱边泄漏被并成 1 段；通过分析才认识到这是区域生长的固有局限，进而把测试数据改为空间分离的两平面，并在报告中如实讨论。这让我体会到：AI 能加速实现，但对结果是否合理的判断必须由人来把关。

九、 实习日志

2026 年 5 月 26 日星期二：选题与调研

确定选题 56 (3D 点云目标分割)。调研区域生长、法向量估计、KD 树等方法，决定不依赖 PCL，核心算法自行实现。

2026 年 6 月 3 日星期三：核心算法

完成 Vec3、PointCloud、KdTree、PCA 法向量、区域生长与由粗到细分割，并写好命令行工具与单元测试，全部测试通过。

2026 年 6 月 8 日星期一：图形界面

基于 GLFW + OpenGL + ImGui 实现三维交互界面：分段着色、迭代层回放、三视图切换、参数实时调节，并截图用于报告。

2026 年 6 月 10 日星期三：测试与报告

完成开发，分析棱边泄漏局限，撰写实习报告并用 XeLaTeX 排版定稿。

十、 实习总结

本次实践完整实现了选题 56 的全部要求：读取并绘制 3D 点云、用区域生长完成考虑表面朝向、位置分布、连通性与一致性度量的分割、由粗到细的迭代细分、以及分色显示与各阶段回放。在工程上，我把数据、算法与显示按面向对象思想拆成相互解耦的类与模块，核心库不依赖界面，因此既能命令行批处理也能交互演示，并用 69 项单元测试保证了正确性。

收获主要有三点：一是真正理解了点云处理“近邻—法向—分割”的完整链条，以及 PCA 与区域生长背后的几何含义；二是体会到面向对象的“解耦”在测试和复用上的价值——把算法与 OpenGL 分开后，算法可以脱离图形环境单独验证；三是认识到任何算法都有适用边界（如区域生长的棱边泄漏），需要用合适的测试去暴露并诚实地

分析。不足之处在于分割算法仍较基础，对噪声和复杂真实点云的鲁棒性有限，后续可引入边界检测、超体素或基于图割的方法进一步提升效果。

A. 附录: 程序设计及报告要求

1. 代码量

(1) 核心代码行数

核心代码应达到 500 行以上 (含必要的空行与注释)。若使用 AI 辅助编程, 总代码量不少于 1000 行。

2. 功能方向

(1) 用户界面

推荐使用控制台完成, UI 界面并非必需。

(2) 可选项目示例

- 图书管理系统 (核心类: Book、User、Library、BorrowRecord 等);
- 学生成绩管理系统 (核心类: Student、Course、ScoreManager 等)。

(3) 测试数据要求

测试数据要充分, 需考虑输入合法性校验、边界值处理等多种出错情形。

凡选择管理系统类题目, 必须要有文件读写操作, 文件中初始数据不得少于 100 个数据元素 (复制或随机生成), 且需要对相关数据的有效性进行合理验证; 选择游戏、计算器、算法动态演示等题目的同学需要设计简易的交互界面。

3. 类与文件要求

(1) 类设计

类的数量不少于 4 个, 保证每个类有明确职责, 体现封装及简单的继承关系。

(2) 多文件结构

文件数量不少于 5 个, 每个文件不超过 200 行 (main 函数所在文件除外)。推荐使用头文件与源文件分离的组织方式, 例如:

```
Book.h / Book.cpp
Library.h / Library.cpp
main.cpp
```

4. 报告与设计要求

(1) UML 图

绘制简单的 UML 类图, 标明类的属性与方法, 以及类之间的简单关系 (如继承、关联)。

(2) 报告格式

撰写格式规范, 段落合理, 图片需加图注并编号。报告应重点体现:

- 代码编写思路；
- 类的设计理由；
- 遇到的问题及解决过程。

避免使用形式化的模板套话。

(3) 报告篇幅

报告核心内容不少于 5 页，附录中的核心代码不超过 3 页。

B. 附录：开发工具与资源要求

1. 编程语言

C++

2. 代码管理

Gitee 或 GitHub（推荐使用，但非必需）

3. 编程工具推荐

Codeblocks、Visual Studio Code、Visual Studio 等。

4. 图形界面（界面非必需，可无界面）

(1) SFML（推荐）

简单多媒体库。耿楠教授录制教程：<https://bilibili.com/video/BV1Ev411N7kG/>

(2) Qt（推荐）

孙海天老师录制教程：<https://www.bilibili.com/video/BV1k8411Z7fk/>

(3) MFC / EasyX / wxWidgets

若使用 C 风格代码，将是扣分项。

5. 其他相关开源库（选用）

(1) PCL 点云处理库

(2) OpenGL 图形库

(3) OpenCV 计算机视觉库

6. 论文撰写

(1) L^AT_EX（推荐）

•耿楠教授录制教程：

https://www.bilibili.com/video/BV15b411j7Au/?vd_source=57a4513a9d3e5a7d74c46a2170adfaf7

•LaTeX 安装教程：

https://www.bilibili.com/video/BV1tg4y1B7f3/?spm_id_from=333.337.search-card.all.click&vd_source=57a4513a9d3e5a7d74c46a2170adfaf7

•报告模板（作业管理系统 / 相关材料 / 附件 2-实习报告 (LaTeX 模板)）

(2) WPS / Word

C. 附录：核心代码清单

本附录列出核心算法库的关键源文件。完整工程（含图形界面、命令行工具与单元测试）见随附源码仓库，目录结构与构建方法见仓库根目录的 README.md。

1. KD 树 (k 近邻查询)

```

1 #include "pcseg/KdTree.h"
2
3 #include <algorithm>
4 #include <cmath>
5
6 namespace pcseg {
7
8 void KdTree::build(const std::vector<Vec3>& points) {
9     points_ = &points;
10    nodes_.clear();
11    root_ = -1;
12    if (points.empty()) return;
13
14    nodes_.reserve(points.size());
15    // 准备一个下标数组，建树时只对下标做划分，不真正移动点数据
16    std::vector<int> indices(points.size());
17    for (int i = 0; i < (int)points.size(); ++i) indices[i] = i;
18
19    root_ = buildRecursive(indices, 0, (int)indices.size(), 0);
20 }
21
22 // 取出坐标的第 axis 个分量
23 static inline float axisValue(const Vec3& p, int axis) {
24     if (axis == 0) return p.x;
25     if (axis == 1) return p.y;
26     return p.z;
27 }
28
29 int KdTree::buildRecursive(std::vector<int>& indices, int first, int last, int
    depth) {
30     if (first >= last) return -1;
31
32     int axis = depth % 3;          // 每往下一层换一个分割轴：x -> y -> z -> x ...
33     int mid = (first + last) / 2;
34
35     // 用 nth_element 把中位数那个点放到 mid 位置（按当前轴的坐标排序），
36     // 左边都比它小、右边都比它大。这样建出来的树比较平衡。
37     const std::vector<Vec3>& pts = *points_;

```

```

38     std::nth_element(indices.begin() + first, indices.begin() + mid, indices.begin() + last,
39                     [&](int a, int b) {
40                         return axisValue(pts[a], axis) < axisValue(pts[b], axis);
41                     });
42
43     Node node;
44     node.pointIndex = indices[mid];
45     node.axis = axis;
46
47     int nodeIdX = (int)nodes_.size();
48     nodes_.push_back(node);    // 先占位，子树下标随后回填
49
50     int leftChild = buildRecursive(indices, first, mid, depth + 1);
51     int rightChild = buildRecursive(indices, mid + 1, last, depth + 1);
52     nodes_[nodeIdX].left = leftChild;
53     nodes_[nodeIdX].right = rightChild;
54     return nodeIdX;
55 }
56
57 std::vector<int> KdTree::kNearest(const Vec3& query, int k) const {
58     std::vector<int> result;
59     if (root_ < 0 || k <= 0) return result;
60
61     // 用一个"最大堆"维护当前找到的最近 k 个点：堆顶是这 k 个里最远的那个，
62     // 这样遇到更近的点时直接替换堆顶即可。元素是（距离平方，点下标）。
63     std::vector<std::pair<float, int>> heap;
64     heap.reserve(k + 1);
65     searchRecursive(root_, query, k, heap);
66
67     // 堆里就是结果，按距离从近到远排序后返回下标
68     std::sort(heap.begin(), heap.end());
69     result.reserve(heap.size());
70     for (const auto& item : heap) result.push_back(item.second);
71     return result;
72 }
73
74 void KdTree::searchRecursive(int nodeIdX, const Vec3& query, int k,
75                             std::vector<std::pair<float, int>>& heap) const {
76     if (nodeIdX < 0) return;
77
78     const Node& node = nodes_[nodeIdX];
79     const Vec3& p = (*points_)[node.pointIndex];
80

```

```
81 // 当前点到查询点的距离, 先尝试放进 k 近邻堆
82 float d2 = distanceSquared(query, p);
83 if ((int)heap.size() < k) {
84     heap.push_back({d2, node.pointIndex});
85     std::push_heap(heap.begin(), heap.end()); // 默认大顶堆, 堆顶是最大距离
86 } else if (d2 < heap.front().first) {
87     std::pop_heap(heap.begin(), heap.end());
88     heap.back() = {d2, node.pointIndex};
89     std::push_heap(heap.begin(), heap.end());
90 }
91
92 // 决定先搜哪一侧: 查询点在分割面的哪一边就先搜哪边
93 float diff = axisValue(query, node.axis) - axisValue(p, node.axis);
94 int nearSide = (diff < 0) ? node.left : node.right;
95 int farSide = (diff < 0) ? node.right : node.left;
96
97 searchRecursive(nearSide, query, k, heap);
98
99 // 另一侧要不要搜? 只有当"分割面距离"比当前第 k 近还要近时,
100 // 另一侧才可能有更近的点, 否则可以剪枝跳过。
101 bool needFar = (int)heap.size() < k || (diff * diff) < heap.front().first;
102 if (needFar) {
103     searchRecursive(farSide, query, k, heap);
104 }
105 }
106
107 } // namespace pcseg
```

代码 3 KdTree.cpp

2. PCA 法向量与曲率估计

```
1 #include "pcseg/Normals.h"
2 #include "pcseg/Eigen3x3.h"
3
4 namespace pcseg {
5
6 void estimateNormals(PointCloud& cloud, const KdTree& tree, int k) {
7     const std::size_t n = cloud.points.size();
8     cloud.normals.assign(n, Vec3(0, 0, 0));
9     cloud.curvature.assign(n, 0.0f);
10    if (n == 0) return;
11
12    // 邻域太大没意义, 至少要 3 个点才能定一个面
```

```

13  if (k < 3) k = 3;
14
15  for (std::size_t i = 0; i < n; ++i) {
16      std::vector<int> nb = tree.kNearest(cloud.points[i], k);
17      if (nb.size() < 3) {
18          // 邻居太少，给个默认向上的法向量
19          cloud.normals[i] = Vec3(0, 0, 1);
20          cloud.curvature[i] = 0.0f;
21          continue;
22      }
23
24      // 1) 邻域质心
25      Vec3 mean(0, 0, 0);
26      for (int idx : nb) mean += cloud.points[idx];
27      mean = mean / static_cast<float>(nb.size());
28
29      // 2) 协方差矩阵（对称，只需算 6 个元素）
30      float c00 = 0, c01 = 0, c02 = 0, c11 = 0, c12 = 0, c22 = 0;
31      for (int idx : nb) {
32          Vec3 d = cloud.points[idx] - mean;
33          c00 += d.x * d.x;
34          c01 += d.x * d.y;
35          c02 += d.x * d.z;
36          c11 += d.y * d.y;
37          c12 += d.y * d.z;
38          c22 += d.z * d.z;
39      }
40      float inv = 1.0f / static_cast<float>(nb.size());
41      c00 *= inv; c01 *= inv; c02 *= inv; c11 *= inv; c12 *= inv; c22 *= inv;
42
43      // 3) 特征值分解：最小特征值的特征向量就是法向量
44      SymEigen e = symmetricEigen(c00, c01, c02, c11, c12, c22);
45      Vec3 normal = e.vectors[0];
46
47      // 4) 曲率 = 最小特征值 / 三个特征值之和
48      float sum = e.values[0] + e.values[1] + e.values[2];
49      float curv = (sum > 1e-12f) ? (e.values[0] / sum) : 0.0f;
50
51      // 让法向量大致朝向 +z（统一一下方向，纯粹是为了显示好看；
52      // 区域生长里比较法向时用的是夹角的绝对值，不依赖这个符号）。
53      if (normal.z < 0) normal = normal * -1.0f;
54
55      cloud.normals[i] = normal.normalized();
56      cloud.curvature[i] = curv;

```

```
57     }
58 }
59
60 } // namespace pcseg
```

代码 4 Normals.cpp

3. 区域生长与由粗到细分割

```
1 #include "pcseg/Segmentation.h"
2
3 #include <algorithm>
4 #include <cmath>
5 #include <queue>
6
7 namespace pcseg {
8
9 std::vector<std::vector<int>>> buildKnnAdjacency(const PointCloud& cloud,
10                                                const KdTree& tree, int k) {
11     const int n = (int)cloud.points.size();
12     std::vector<std::vector<int>>> adjacency(n);
13     for (int i = 0; i < n; ++i) {
14         // kNearest 通常会把点自己作为第一个返回, 扩张时无所谓, 保留即可
15         adjacency[i] = tree.kNearest(cloud.points[i], k);
16     }
17     return adjacency;
18 }
19
20 std::vector<int> regionGrow(const PointCloud& cloud,
21                            const std::vector<std::vector<int>>>& adjacency,
22                            const std::vector<char>& allowed,
23                            float smoothnessRad,
24                            float curvatureThreshold,
25                            int minClusterSize,
26                            int& segCountOut) {
27     const int n = (int)cloud.points.size();
28     std::vector<int> labels(n, -1);
29     const float cosThreshold = std::cos(smoothnessRad); // 用余弦比较, 省去每次
30                                                         // acos
31     // 把子集内的点按曲率从小到大排序, 越平坦的越先当种子 (区域生长的经典做法)
32     std::vector<int> order;
33     order.reserve(n);
34     for (int i = 0; i < n; ++i) {
```

```

35     if (allowed[i]) order.push_back(i);
36 }
37 std::sort(order.begin(), order.end(), [&](int a, int b) {
38     return cloud.curvature[a] < cloud.curvature[b];
39 });
40
41 int segId = 0;
42 for (int seed : order) {
43     if (labels[seed] != -1) continue; // 已经归到某个段了
44
45     // 从这个种子开始，广度优先地向外扩张出一个段
46     std::queue<int> q;
47     q.push(seed);
48     labels[seed] = segId;
49
50     while (!q.empty()) {
51         int cur = q.front();
52         q.pop();
53         const Vec3& nCur = cloud.normals[cur];
54
55         for (int nb : adjacency[cur]) {
56             if (!allowed[nb] || labels[nb] != -1) continue;
57
58             // 当前点与邻居法向量的夹角，用 |cos| 比较（忽略法向正反方向）
59             float c = std::fabs(dot(nCur, cloud.normals[nb]));
60             if (c < cosThreshold) continue; // 夹角太大，不属于同一光滑面
61
62             labels[nb] = segId; // 并入当前段
63             // 只有足够平坦的点才继续作为新的扩张前沿，避免越过棱边
64             if (cloud.curvature[nb] < curvatureThreshold) {
65                 q.push(nb);
66             }
67         }
68     }
69     ++segId;
70 }
71
72 // 统计每个段的点数，丢弃太小的段（标回 -1），再把段号压缩成连续的 0..K-1
73 std::vector<int> count(segId, 0);
74 for (int i = 0; i < n; ++i) {
75     if (labels[i] >= 0) count[labels[i]]++;
76 }
77 std::vector<int> remap(segId, -1);
78 int kept = 0;

```



```
79     for (int s = 0; s < segId; ++s) {
80         if (count[s] >= minClusterSize) remap[s] = kept++;
81     }
82     for (int i = 0; i < n; ++i) {
83         if (labels[i] >= 0) labels[i] = remap[labels[i]];
84     }
85
86     segCountOut = kept;
87     return labels;
88 }
89
90 SegResult segment(const PointCloud& cloud, const KdTree& tree, const SegParams&
    params) {
91     SegResult result;
92     const int n = (int)cloud.points.size();
93     if (n == 0) return result;
94
95     const double deg2rad = 3.14159265358979323846 / 180.0;
96     std::vector<std::vector<int>> adjacency = buildKnnAdjacency(cloud, tree,
    params.k);
97
98     int levels = std::max(1, params.levels);
99
100    // 第 0 层：整片点云都参与，用最粗（最大）的夹角阈值
101    std::vector<char> allowAll(n, 1);
102    int count0 = 0;
103    float angle0 = (float)(params.coarseSmoothnessDeg * deg2rad);
104    std::vector<int> labels0 = regionGrow(cloud, adjacency, allowAll, angle0,
    params.curvatureThreshold,
105    params.minClusterSize, count0);
106
107    result.levelLabels.push_back(labels0);
108    result.segmentCount.push_back(count0);
109
110    // 之后每一层：在上一层每个段的内部，用更小的夹角阈值继续细分
111    for (int L = 1; L < levels; ++L) {
112        // 夹角阈值从 coarse 线性过渡到 fine
113        float t = (float)L / (float)(levels - 1);
114        float deg = params.coarseSmoothnessDeg +
115            t * (params.fineSmoothnessDeg - params.coarseSmoothnessDeg);
116        float angle = (float)(deg * deg2rad);
117
118        const std::vector<int>& prev = result.levelLabels.back();
119        int prevCount = result.segmentCount.back();
120
```

```
121     std::vector<int> labels(n, -1);
122     int globalSeg = 0;
123     // 对上一层的每一个段，单独再做一次区域生长
124     for (int p = 0; p < prevCount; ++p) {
125         std::vector<char> allowed(n, 0);
126         for (int i = 0; i < n; ++i) {
127             if (prev[i] == p) allowed[i] = 1;
128         }
129         int subCount = 0;
130         std::vector<int> sub = regionGrow(cloud, adjacency, allowed, angle,
131                                         params.curvatureThreshold,
132                                         params.minClusterSize, subCount);
133         // 把这个父段细分出来的局部段号，平移到全局段号上
134         for (int i = 0; i < n; ++i) {
135             if (sub[i] >= 0) labels[i] = globalSeg + sub[i];
136         }
137         globalSeg += subCount;
138     }
139     result.levelLabels.push_back(labels);
140     result.segmentCount.push_back(globalSeg);
141 }
142
143 return result;
144 }
145
146 } // namespace pcseg
```

代码 5 Segmentation.cpp